

Android UI Development

Android Studio

Widget

Layout



Spinners

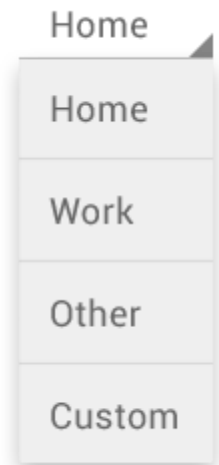
- Spinners provide a quick way to select one value from a set
- Touching the spinner displays a dropdown menu with all other available values

```
<Spinner  
    android:id="@+id/planets_spinner"  
    android:layout_width="fill_parent"  
    android:layout_height="wrap_content" />
```

} layout

```
<?xml version="1.0" encoding="utf-8"?>  
<resources>  
    <string-array name="my_array">  
        <item>Home</item>  
        <item>Work</item>  
        <item>Other</item>  
        <item>Custom</item>  
    </string-array>  
</resources>
```

} values/strings.xml



Spinners

With an array, you can use the following code in your Activity to supply the spinner with the array using an instance of **ArrayAdapter**:

```
Spinner spinner = (Spinner) findViewById(R.id.spinner);

// Create an ArrayAdapter using the string array and a default
// spinner layout
ArrayAdapter<CharSequence> adapter =
    ArrayAdapter.createFromResource(this,
        R.array.my_array, android.R.layout.simple_spinner_item);

// Specify the layout to use when the list of choices appears
adapter.setDropDownViewResource(android.R.layout.simple_spinner_
    dropdown_item);

// Apply the adapter to the spinner
spinner.setAdapter(adapter);
```

Code to get spinner values in Android

When the user selects an item from the drop-down, the Spinner object receives the currently selected value.

```
Spinner mySpinner = (Spinner) findViewById(R.id.my_spinner);  
String value = spinner.getSelectedItem().toString();
```

Responding to User Selections

- When the user selects an item from the drop-down, the Spinner object receives an **on-item-selected event**.

```
public class SpinnerActivity extends Activity
    implements OnItemSelectedListener {

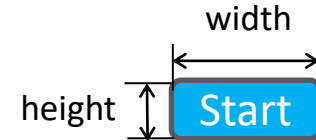
    ...

    public void onItemSelected(AdapterView<?> parent, View view,
        int pos, long id) {
        // An item was selected. You can retrieve the selected
        // item using parent.getItemAtPosition(pos)
    }

    public void onNothingSelected(AdapterView<?> parent) {
        // Another interface callback
    }
}
```

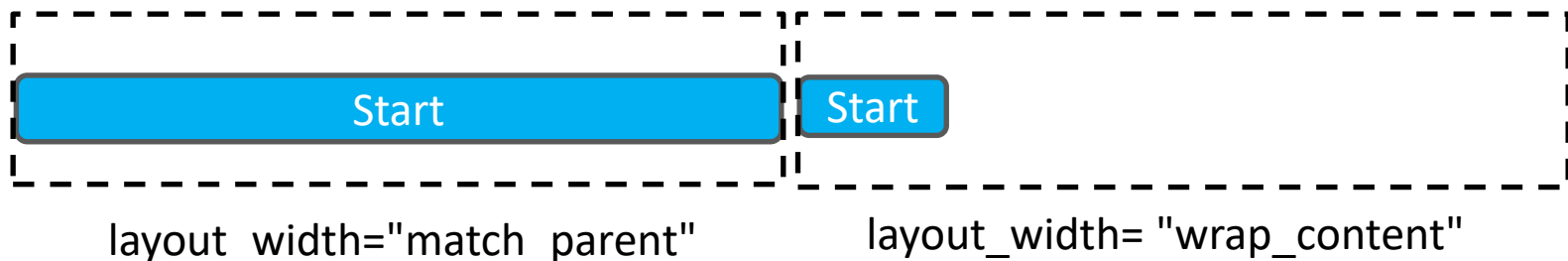
Specifying Size of View

- Many view attributes, including the two required attributes:
- **layout_width**
- **layout_height**



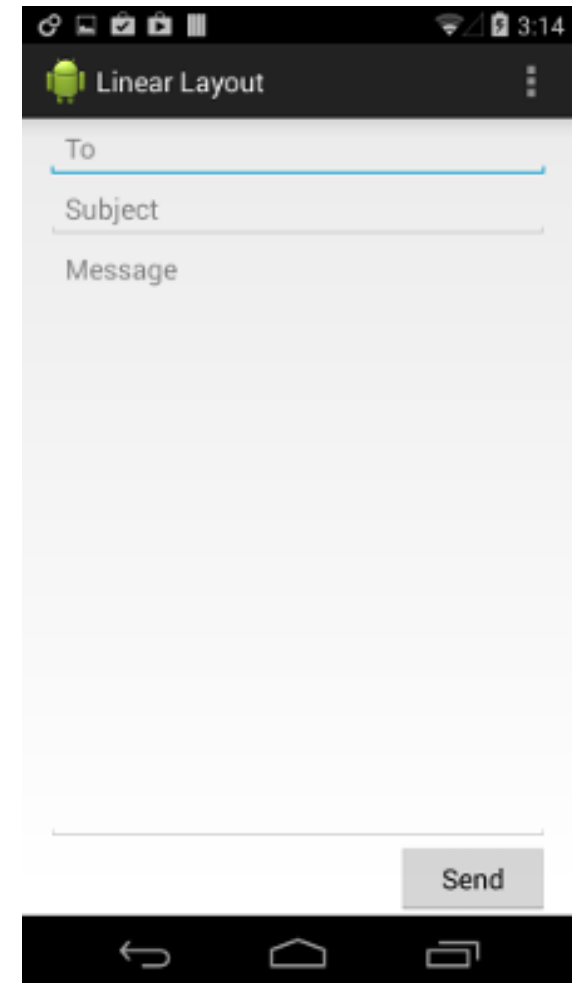
So that layout knows how to size a view

```
android:layout_height="wrap_content"  
android:layout_width="match_parent"    or  
android:layout_width="100dp"
```



Linear Layout

- **LinearLayout** is a view group that aligns all children in a single direction, vertically or horizontally.
- You can specify the layout direction with the `android:orientation` attribute.
- All children of a LinearLayout are stacked one after the other
 - a vertical list will only have one child per row, no matter how wide they are
 - a horizontal list will only be one row high



<https://developer.android.com/guide/topics/ui/layout/linear>

Key Attributes

▪ Orientation

Should the layout be a column or a row?

- "horizontal" for a row
- "vertical" for a column.

▪ Fill model

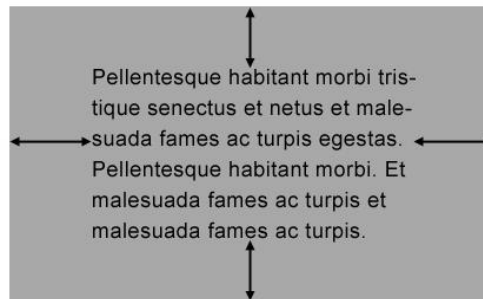
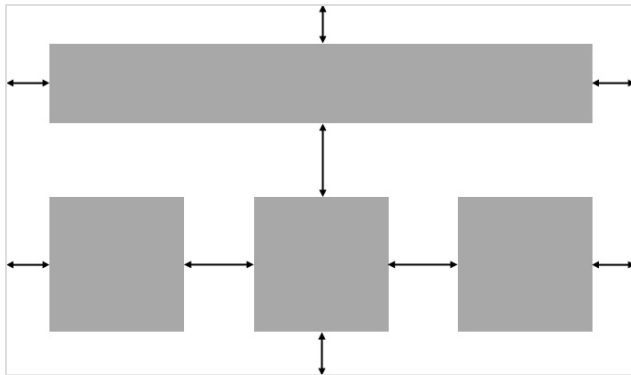
MATCH_PARENT: the view wants to be as big as its parent

WRAP_CONTENT: the view wants to be just large enough to fit its own internal content

Key Attributes

▪ Padding/margin

Margin means the space outside the view, however padding means the space inside the view.

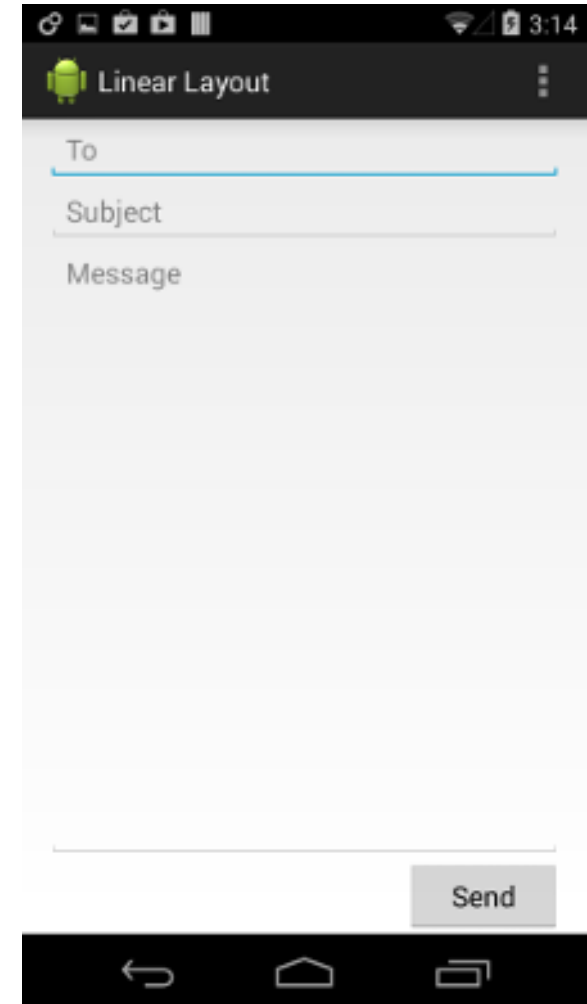


▪ Weight

`android:layout_weight` attribute assigns an "importance" value to a view in terms of how much space it should occupy on the screen.

LinearLayout

```
<LinearLayout ...>
  <EditText
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="@string/to" />
  <EditText
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="@string/subject" />
  <EditText
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="1"
    android:gravity="top"
    android:hint="@string/message" />
  <Button
    android:layout_width="100dp"
    android:layout_height="wrap_content"
    android:layout_gravity="right"
    android:text="@string/send" />
</LinearLayout>
```



Relative Layout

- RelativeLayout is a view group that displays child views in relative positions.
- The position of each view can be specified as
 - relative to sibling elements (such as to the left-of or below another view)
 - in positions relative to the parent RelativeLayout area (such as aligned to the bottom, left or center).



<https://developer.android.com/guide/topics/ui/layout/relative>

Nested Layout

<RelativeLayout

```
xmlns:android="http://schemas.android.com/apk/res/android"  
android:layout_width="match_parent"  
android:layout_height="match_parent">
```

<TextView

```
    android:id="@+id/name_title"  
    android:text="@string/name">
```

.....

</EditText>

.....

<LinearLayout

```
    android:layout_below="@+id/session">
```

.....

<CheckBox

```
    android:id="@+id/checkbox_morning"  
    android:text="@string/morning"
```

..... />

<CheckBox

..... />

</LinearLayout>

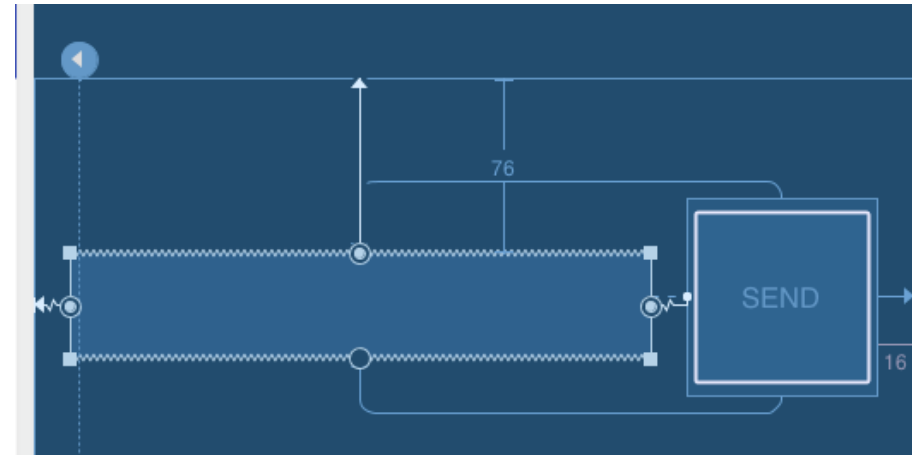
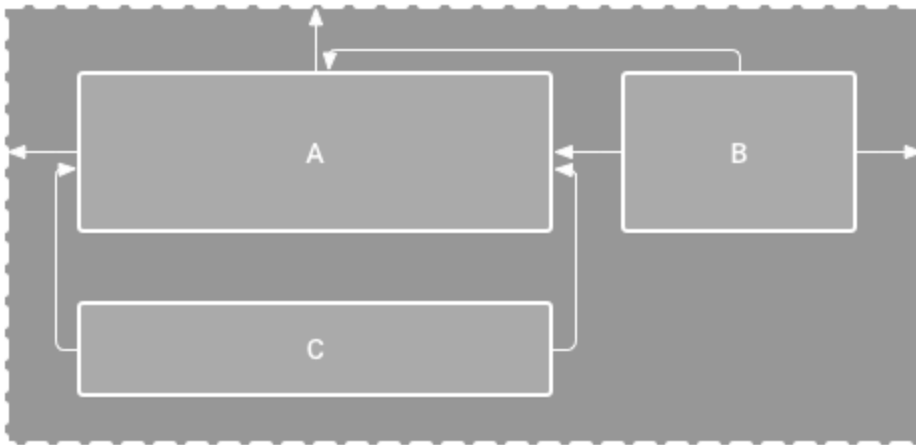
</RelativeLayout>

Views

Views

Constraint Layout

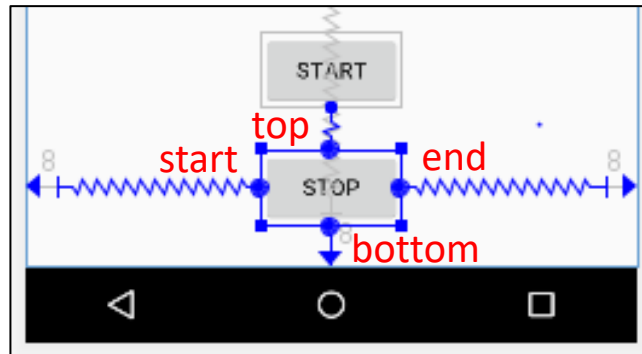
- **Note:** For better performance and tooling support, you should instead build your layout with `ConstraintLayout`.
- Easier to use with Android Studio's Layout Editor
 - Default layout for a new layout



- Short talk: <https://youtu.be/XamMbnzl5vE?t=45>
- Deep Dive: <https://www.youtube.com/watch?v=P9Zstbk0IPw>

ConstraintLayout

- Similar to RelativeLayout
 - All views are laid out according to relationships with others
- At least one **horizontal** and one **vertical** constraint



<Button

android:id="@+id/stopButton"

...

android:text="Stop"

app:layout_constraintStart_toStartOf="parent"

app:layout_constraintEnd_toEndOf="parent"

app:layout_constraintTop_toBottomOf="@+id/startButton"

app:layout_constraintBottom_toBottomOf="parent" />

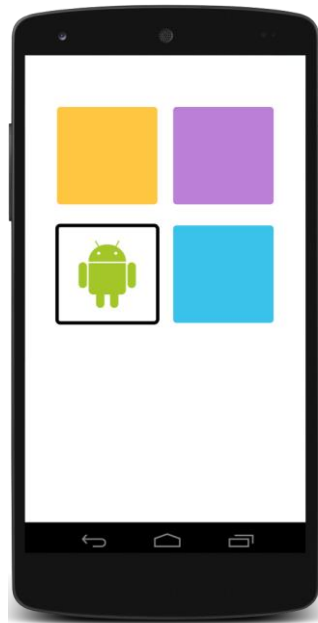
Events

- Android uses the Java event model with additional mobile events
 - Event listener: interface for specific type of event
 - Event handler: registered callback method to handle the event
- <https://developer.android.com/develop/ui/views/touch-and-input/input-events>

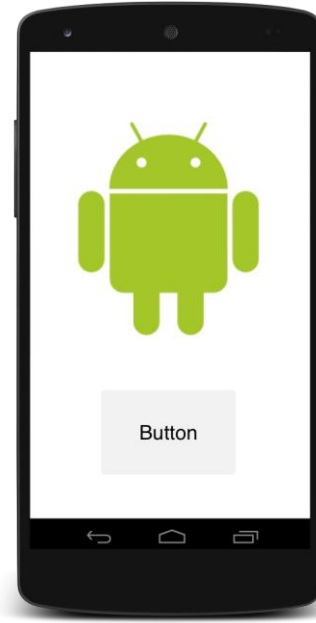
Event Listener	Event Handler	Type of event
OnClickListener()	onClick()	Touch, click
OnLongClickListener()	onLongClick()	Press and hold
onTouchListener()	onTouch()	Generic touch events can be used for touch_up second_touch

Activities

- A standard application component is an Activity
 - Typically represents a single screen
 - Main entry point (equivalent to main() method)
 - For most purposes, this is your application class



Activity 1



Activity 2

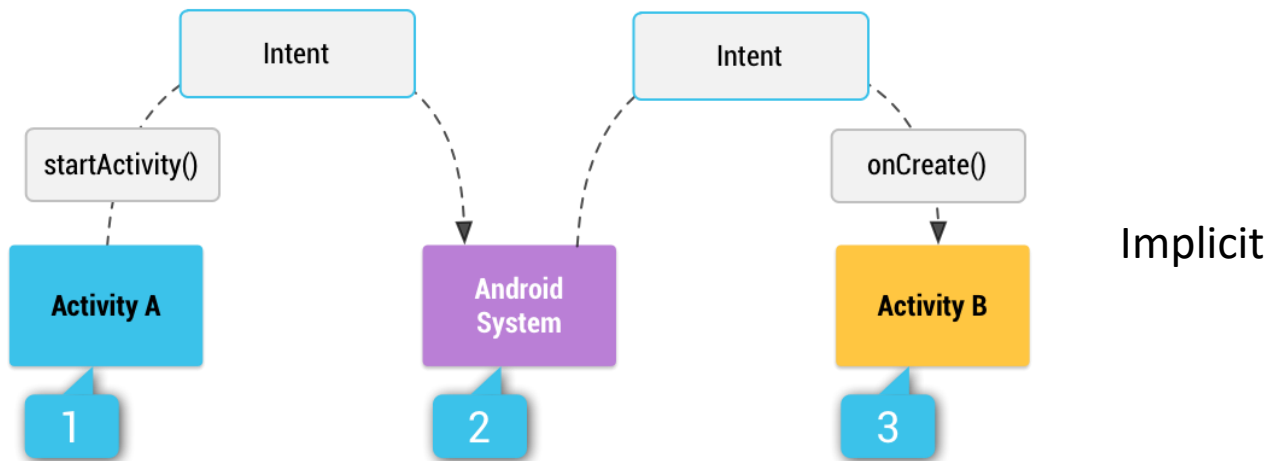
Intents

- An **Intent** is a messaging object you can use to request an action from another app component
 - E.g. starting an activity, starting a service
- We use **intents** to pass data between activities

<https://developer.android.com/guide/components/intents-filters.html>

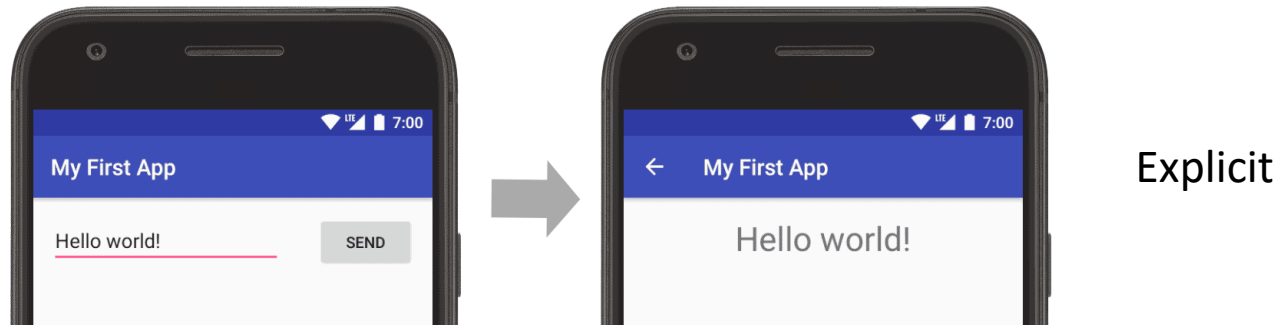
Intents

- Implicit (capabilities): do not name a specific component, but instead declare a general action to perform
 - Example: Accessing a camera



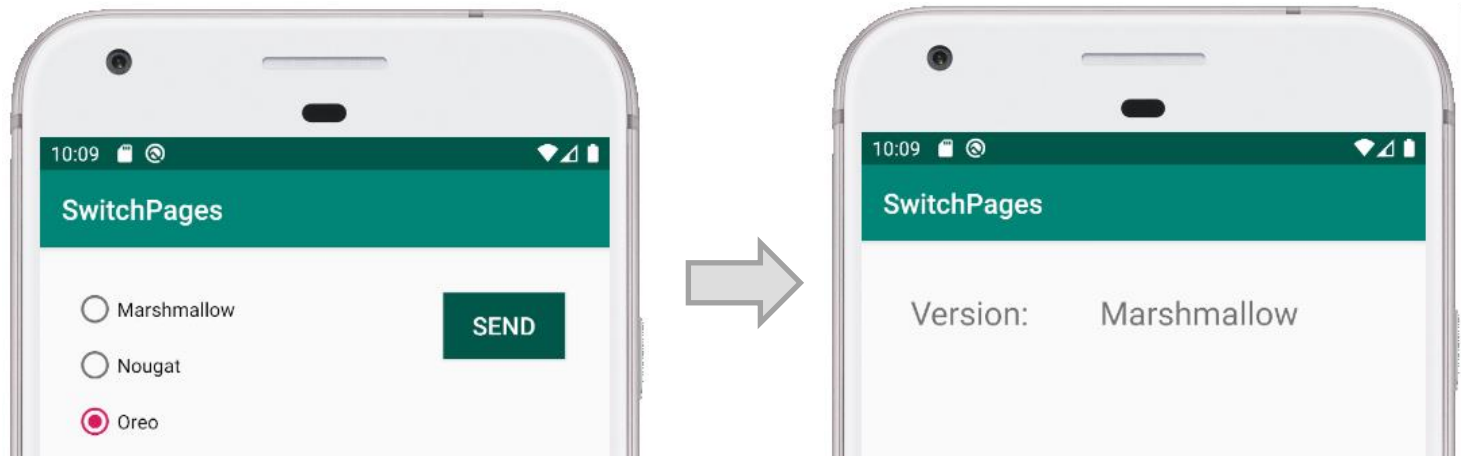
Intents

- Explicit (named activity): specify which application will satisfy the intent
 - Example: you might start a new activity within your app in response to a user action



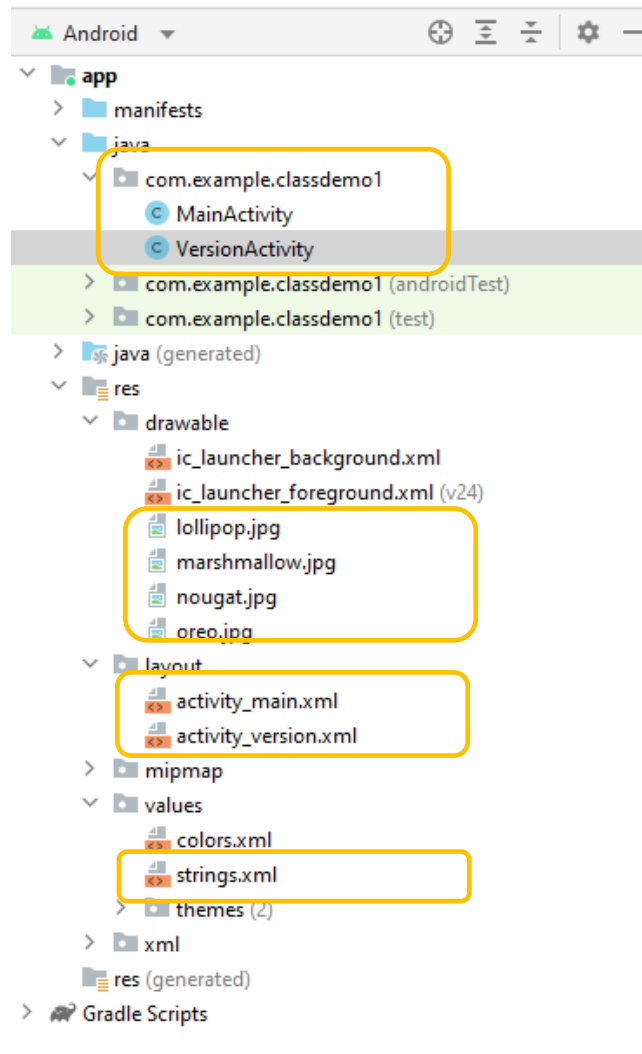
Multiple activities

- Usually, one view per activity
- Only one activity is running at any given time
- How can we share data between views?

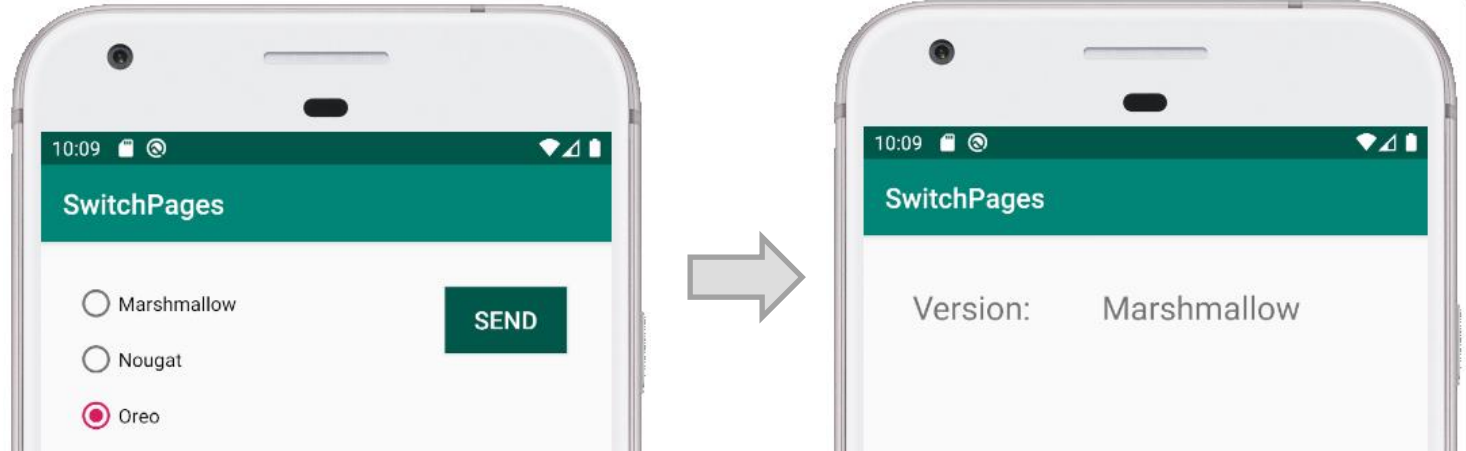


Add another activity

- File → New → Activity → Empty Activity



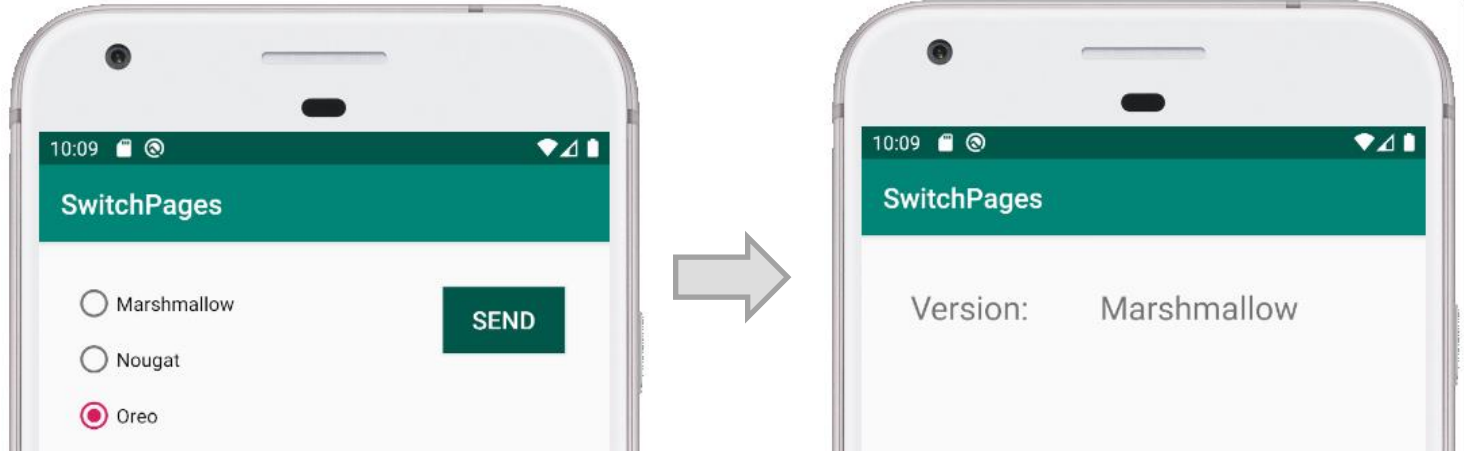
Intent: Start Another Activity



```
int version = 0;
switch (radio_id){
    case R.id.radioButton1:
        version = 6;
        break;
    ...
}
```

```
Intent intent = new Intent(this, SecondActivity.class);
intent.putExtra("version", version);
startActivity(intent);
```

Start Another Activity



`// Get the Intent that started this activity
//and extract the value in int
Intent intent = getIntent();
int v = intent.getIntExtra("version",0);

// Set the string match to the value
TextView label = findViewById(R.id.version_txt);
switch(v){
 case 6:
 label.setText(R.string.v6);`

Sending data between activities

- Android Bundle also used to pass data between activities
- To send data:

```
Intent intent = new Intent(this, SecondActivity.class);
```

```
Bundle bundle = new Bundle();
```

```
bundle.putString("fullname", fullname);
```

```
bundle.putLong("phoneNumber", phone);
```

```
bundle.putDouble("age", ageDouble);
```

```
bundle.putBoolean("married", isMarried);
```

```
intent.putExtras(bundle);
```

```
startActivity(intent);
```


Sending data between activities

- To receive data:

```
Intent intent = getIntent();  
Bundle bundle = intent.getExtras();
```

```
String nameString = bundle.getString("fullname");  
long phoneNumberLong = bundle.getLong("phoneNumber");  
double ageDouble = bundle.getDouble("age");  
boolean isMarriedBoolean = bundle.getBoolean("married");
```

Questions?